

Algorithmique 1 — TD 1

Tests unitaires & spécification de fonction · UE Info1.Algo1 · L1 Informatique, UPS · Semestre 1 2026-2027

Les exercices sont présentés en deux temps : Partie 1 — Énoncés (les questions et, s'il est fourni par l'énoncé, le code à tester, sans réponse, pour pouvoir refaire le TD) puis, après un saut de page, Partie 2 — Corrigé (raisonnement complet, fonctions écrites et jeux de test vérifiés en les exécutant réellement).

Rappels — principe des tests unitaires

- Un **jeu de test** spécifie, pour une fonction et des **paramètres** donnés, la valeur de **retour** ou l'**effet** (dans le cas d'une liste modifiée en place) attendus.
- Les jeux de test doivent couvrir le **maximum de cas possibles** : cas particuliers et cas généraux.
- L'**objectif** est de mettre en défaut le code testé, pas seulement de vérifier qu'il marche sur un exemple facile.
- On écrit les jeux de test en **boîte noire** (avant d'écrire la fonction testée) ou en **boîte blanche** (le code de la fonction testée est alors fourni).

Assertions

```
assert <expression booléenne>, "message d'erreur"
```

Le message d'erreur est affiché si l'expression booléenne est évaluée à `False`, sinon l'exécution est silencieuse.

Fonction de test

Rassemble les jeux de test (assertions) associés à une fonction donnée. Nommage : `test_<nom fonction> tester`.

Dans les exercices 1, 3, 4 et 5 (boîte noire), on ne demande **pas** d'écrire la fonction testée elle-même — uniquement les jeux de test et la fonction de test.

Rappels — spécification de fonction

- Le **type** des paramètres et de la (des) valeur(s) de retour.
- La **pré-condition** : expression booléenne devant être vérifiée par les paramètres pour que le problème puisse être résolu.
- La **post-condition** : expression booléenne devant être vérifiée par la (les) valeur(s) de retour.

Modèle pour l'instrumentation :

```
def <nom de la fonction>(<param1>, <param2>):  
    assert ..., 'Pre-condition'  
    ... # CODE DE LA FONCTION  
    assert ..., 'Post-condition'  
    return ...
```

- Pour garantir que la post-condition est cohérente avec le problème posé, les **paramètres de la fonction ne doivent pas être modifiés**.
- Pour les **fonctions auxiliaires** (qui servent justement aux tests de propriétés), aucune pré-condition / post-condition n'est demandée.

Partie 1 — Énoncés

EXERCICES : TESTS UNITAIRES

Exercice 1 — produit des éléments

On souhaite tester en boîte noire la fonction `produit_elements` qui accepte en paramètre une liste d'entiers et retourne le produit de ses éléments.

1) Écrire 6 jeux de tests distincts correspondant aux cas suivants :

- Liste vide.
- Singleton.
- Listes de longueurs toutes différentes et supérieures à 3 contenant :
 - Uniquement des nombres strictement positifs.
 - Uniquement des nombres strictement négatifs.
 - Des nombres non nuls (négatifs et positifs).
 - Exactement un zéro.

2) Écrire la fonction de test correspondante.

Exercice 2 — nombre de chiffres

On souhaite tester en boîte blanche la fonction `nombre_chiffres` qui accepte en paramètre un entier quelconque et retourne son nombre de chiffres. Son code est le suivant :

```
def nombre_chiffres(n):  
    if n < 0:  
        n = -n  
    k = 1  
    while n >= 10:  
        n //= 10  
        k += 1  
    return k
```

1) Écrire 9 (c'est-à-dire $1 + 2 \times 4$) jeux de tests distincts correspondant aux cas suivants :

- Zéro.
- Condition du if fausse ou vraie et, dans chacun des cas :
 - Pas d'itération.
 - Exactement 1 itération.
 - Exactement 4 itérations.
 - Plus de 10 itérations.

2) a) Donner les 2 entiers consécutifs tels qu'il y ait 2 itérations pour le premier et 3 itérations pour son successeur.

b) Écrire les 2 assertions de test correspondantes.

Exercice 3 — insertion ordonnée

On souhaite tester la fonction `insertion_ordonnee` qui accepte en paramètres `liste` (une liste d'entiers triée par ordre croissant, au sens large) et `valeur` (une valeur entière). La fonction insère `valeur` dans une **copie** de `liste` de telle façon que la nouvelle liste reste triée, et retourne la liste ainsi créée.

Exemple : si `liste` vaut `[7,11]` et `valeur` vaut `9`, l'insertion a lieu à l'indice `1`, et la liste retournée est `[7,9,11]`.

1) a) Écrire 5 jeux de tests distincts correspondant aux cas suivants :

- Liste vide.
- Liste de longueur 4 avec insertion en début, en fin, à une autre position, ou avec une valeur égale à un élément déjà présent (cas « au sens large »).

b) Écrire les lignes de code (avec assertions) nécessaires à la vérification du dernier jeu de test. On vérifiera en particulier que la fonction ne modifie pas la liste passée en paramètre.

2) On souhaite au contraire que la fonction ait pour effet de modifier la liste passée en paramètre, sans rien retourner. Écrire les lignes de code (avec assertions) nécessaires à la vérification du dernier jeu de test selon ce nouveau principe.

Exercice 4 — valeurs inférieures ★ (intermédiaire)

Écrire les jeux de tests et la fonction de test associés à `nb_valeurs_inferieures(liste, valeur)`, qui détermine et retourne le nombre d'éléments de liste dont la valeur est inférieure ou égale à `valeur`.

Exemple : si liste vaut `[-3,2,8,1,7,11,3,4]` et valeur vaut 3, la fonction doit retourner 4 (valeurs -3, 2, 1, 3).

Exercice 5 — indices ★ (intermédiaire)

Écrire les jeux de tests et la fonction de test associés à `indices_valeur(liste, valeur)`, qui détermine et retourne la liste des indices dans liste où l'on peut trouver `valeur`.

Exemple : si liste vaut `[11,23,67,23,23,83,23,10]` et valeur vaut 23, la fonction doit retourner `[1,3,4,6]`.

EXERCICES : SPÉCIFICATION DE FONCTION

Exercice 6 — lecture d'énoncé

Écrire la spécification des fonctions suivantes :

- 1) La fonction `puissance` calcule et retourne la puissance $p = a^b$ de deux entiers positifs `a` et `b`.
- 2) La fonction `log2_entier` accepte en paramètre un entier `n` strictement positif et retourne l'unique entier `k` tel que $2^k \leq n < 2^{k+1}$.

Exercice 7 — écriture de fonction auxiliaire

On souhaite écrire la pré-condition pour l'exercice 3 (fonction `insertion_ordonnee`).

- 1) Écrire la fonction auxiliaire `est_croissante` qui accepte en paramètre une liste d'entiers `liste`, vérifie que cette liste est triée par ordre croissant (au sens large) et retourne le booléen correspondant.
- 2) En déduire l'assertion de pré-condition de la fonction `insertion_ordonnee`.

(L'écriture de la post-condition dépasse le cadre de ce TD.)

Exercice 8 — comprendre une spécification

On donne la spécification de fonction suivante :

- Type du paramètre : `n` est un entier.
- Type de la valeur de retour : `a` est un entier.
- Pré-condition : $n \geq 0$.
- Post-condition : $a \geq 0$ and $a^2 \leq n < (a+1)^2$.

On répondra dans l'ordre souhaité : écrire une implémentation de cette spécification, puis décrire la nature du problème posé et en déduire un nom pertinent pour la fonction.

Contrainte : ne pas utiliser de type `float` qui limiterait le domaine de validité de la fonction.

Exercice 9 — complément sur les ternaires ★★ (avancé)

On souhaite écrire la fonction `nombre_chiffres` qui accepte en paramètre un entier positif ou nul `n` et retourne son nombre `k` de chiffres (en base 10).

- 1) Écrire la pré-condition de cette fonction.
- 2) a) Écrire la fonction auxiliaire `est_nombre_chiffres(n,k)` qui retourne le booléen indiquant si `k` est le nombre de chiffres (en base 10) de l'entier positif ou nul `n`.

Indications : un nombre entier strictement positif a `k` chiffres si et seulement s'il appartient à l'intervalle $[10^{k-1}; 10^k[$. L'exception à cette propriété est l'entier 0, qui possède 1 chiffre.

- b) En déduire la post-condition de la fonction `nombre_chiffres`.
- 3) Réécrire cette post-condition sous forme d'une expression ternaire (`valeur_vrai if condition else valeur_faux`).
- 4) Écrire la fonction `nombre_chiffres`. Le paramètre `n` ne doit pas être modifié par la fonction.
- 5) (Pour aller plus loin) Généraliser dans le cas des nombres relatifs.

Partie 2 — Corrigé

EXERCICES : TESTS UNITAIRES

Exercice 1 — produit des éléments

Méthode : 6 jeux de test de longueurs toutes différentes (0, 1, 4, 5, 6, 7), pour bien séparer les cas. Les valeurs attendues ont été calculées et vérifiées indépendamment (produit élément par élément) avant d'être recopiées ici.

```
def test_produit_elements():
    assert produit_elements([]) == 1, "liste vide : produit neutre"
    assert produit_elements([5]) == 5, "singleton"
    assert produit_elements([1,2,3,4]) == 24, "longueur 4, tous positifs"
    assert produit_elements([-1,-2,-3,-4,-5]) == -120, "longueur 5, tous negatifs"
    assert produit_elements([-2,3,-1,4,-5,6]) == -720, "longueur 6, non nuls melanges"
    assert produit_elements([1,-2,3,0,-4,5,-6]) == 0, "longueur 7, exactement un zero"
    print("tous les tests sont passes")
```

Le cas « liste vide » attend 1 (élément neutre du produit) et non 0, comme la somme d'une liste vide vaut 0 (élément neutre de l'addition). Le cas « exactement un zéro » est un piège classique : il suffit d'un seul 0 pour annuler tout le produit, quels que soient les autres éléments.

Exercice 2 — nombre de chiffres

1) 9 jeux de tests (le code fourni est exécuté pour vérifier chaque valeur attendue) :

```
def test_nombre_chiffres():
    assert nombre_chiffres(0) == 1, "zero"
    # if faux (n>=0) :
    assert nombre_chiffres(7) == 1, "0 iteration"
    assert nombre_chiffres(42) == 2, "1 iteration"
    assert nombre_chiffres(12345) == 5, "4 iterations"
    assert nombre_chiffres(123456789012) == 12, "plus de 10 iterations"
    # if vrai (n<0) :
    assert nombre_chiffres(-7) == 1, "0 iteration"
    assert nombre_chiffres(-42) == 2, "1 iteration"
    assert nombre_chiffres(-12345) == 5, "4 iterations"
    assert nombre_chiffres(-123456789012) == 12, "plus de 10 iterations"
    print("tous les tests sont passes")
```

(Vérifié en exécutant réellement le code fourni sur ces 9 valeurs : tous les résultats correspondent.)

2) a) On cherche n tel que nombre_chiffres(n) fasse 2 itérations et nombre_chiffres(n+1) en fasse 3. Le nombre d'itérations vaut (nombre de chiffres – 1) ; il faut donc que n passe de 3 chiffres à 4 chiffres, c'est-à-dire n = 999 et n+1 = 1000.

```
# 999 -> 999 -> 99 -> 9 : 2 iterations, k = 1+2 = 3 chiffres
# 1000 -> 1000 -> 100 -> 10 -> 1 : 3 iterations, k = 1+3 = 4 chiffres
```

b) Les 2 assertions correspondantes :

```
assert nombre_chiffres(999) == 3, "999 : 2 iterations, 3 chiffres"
assert nombre_chiffres(1000) == 4, "1000 : 3 iterations, 4 chiffres"
```

Exercice 3 — insertion ordonnée

1) a) 5 jeux de tests. Le 5e cas (valeur égale à un élément déjà présent) couvre la clause « au sens large » de l'énoncé : la liste reste valide qu'on insère avant ou après les éléments égaux, seul le multi-ensemble des valeurs compte.

```
# 1. liste vide : insertion_ordonnee([], 5) -> [5]
# 2. insertion en debut : insertion_ordonnee([3,5,7,9], 1) -> [1,3,5,7,9]
# 3. insertion en fin : insertion_ordonnee([2,4,6,8], 10) -> [2,4,6,8,10]
# 4. insertion au milieu : insertion_ordonnee([3,7,9,11], 8) -> [3,7,8,9,11]
# 5. valeur deja presente: insertion_ordonnee([2,4,4,6], 4) -> [2,4,4,4,6]
```

b) Vérification du 5e jeu de test (avec contrôle de non-modification de la liste d'origine) :

```
liste_originale = [2,4,4,6]
resultat = insertion_ordonnee(liste_originale, 4)
assert resultat == [2,4,4,4,6], "la valeur doit etre inseree en gardant la liste trie"
assert liste_originale == [2,4,4,6], "la liste passee en parametre ne doit pas etre modifiee"
```

2) Version « effet de bord » (modifie liste en place, ne retourne rien) :

```
liste = [2,4,4,6]
resultat = insertion_ordonnee(liste, 4)
assert resultat is None, "cette version ne doit rien retourner"
assert liste == [2,4,4,4,6], "la liste doit etre modifiee en place"
```

Ces deux versions incarnent les deux conventions possibles pour une fonction qui « transforme » une liste : **retourner une copie** (la liste d'origine est intacte, plus sûr mais recopie la mémoire) ou **modifier en place** (plus économe en mémoire, mais effet de bord à documenter clairement).

Exercice 4 — valeurs inférieures ★ (intermédiaire)

```
def test_nb_valeurs_inferieures():
    assert nb_valeurs_inferieures([-3,2,8,1,7,11,3,4], 3) == 4, "exemple de l'enonce"
    assert nb_valeurs_inferieures([], 5) == 0, "liste vide"
    assert nb_valeurs_inferieures([10,20,30], 5) == 0, "aucune valeur ne convient"
    assert nb_valeurs_inferieures([1,2,3], 100) == 3, "toutes les valeurs conviennent"
    assert nb_valeurs_inferieures([5,5,5,2], 5) == 4, "egalite (<=), plusieurs occurrences"
    print("tous les tests sont passes")
```

Les tests 1 et 5 vérifient spécifiquement le « ou égale » de l'énoncé : une implémentation buguée utilisant < au lieu de ≤ les ferait échouer (dans l'exemple de l'énoncé, elle compterait 3 valeurs au lieu de 4).

Exercice 5 — indices ★ (intermédiaire)

```
def test_indices_valeur():
    assert indices_valeur([11,23,67,23,23,83,23,10], 23) == [1,3,4,6], "exemple de l'enonce"
    assert indices_valeur([1,2,3], 9) == [], "valeur absente"
    assert indices_valeur([], 5) == [], "liste vide"
    assert indices_valeur([4,5,6], 5) == [1], "une seule occurrence"
    assert indices_valeur([7,7,7], 7) == [0,1,2], "toutes les positions"
    print("tous les tests sont passes")
```

EXERCICES : SPÉCIFICATION DE FONCTION

Exercice 6 — lecture d'énoncé

1) Fonction puissance :

```
def puissance(a, b):
    assert a >= 0 and b >= 0, 'Pre-condition'
    p = ...
    assert p == a**b, 'Post-condition'
    return p
```

Type : a, b, p sont des entiers. Pré-condition : $a \geq 0$ et $b \geq 0$ (deux entiers positifs). Post-condition : $p = a^b$ (avec la convention usuelle $0^0 = 1$).

2) Fonction log2_entier :

```
def log2_entier(n):
    assert n >= 1, 'Pre-condition'
    k = ...
    assert 2**k <= n < 2**(k+1), 'Post-condition'
    return k
```

Type : n, k sont des entiers. Pré-condition : $n \geq 1$ (strictement positif). Post-condition : $2^k \leq n < 2^{k+1}$ (k est le logarithme en base 2 de n , arrondi à l'entier inférieur ; notée « positif » dans l'énoncé, k peut valoir 0, par exemple pour $n=1$).

Exercice 7 — écriture de fonction auxiliaire

1) Fonction auxiliaire `est_croissante` :

```
def est_croissante(liste):
    for i in range(len(liste)-1):
        if liste[i] > liste[i+1]:
            return False
    return True
```

Vérifié sur plusieurs cas (liste vide et singleton $\rightarrow$ True par convention, liste triée $\rightarrow$ True, liste non triée $\rightarrow$ False).

2) Pré-condition déduite pour `insertion_ordonnee` :

```
assert est_croissante(liste), "liste doit etre triee par ordre croissant (au sens large)"
```

Exercice 8 — comprendre une spécification

Implémentation (sans float, par recherche incrémentale) :

```
def racine_carree_entiere(n):
    assert n >= 0, 'Pre-condition'
    a = 0
    while (a+1)**2 <= n:
        a += 1
    assert a >= 0 and a**2 <= n < (a+1)**2, 'Post-condition'
    return a
```

Vérifié par exécution contre `math.isqrt` sur une large plage de valeurs (0 à 10^6+1 , y compris juste avant/après des carrés parfaits comme 15/16 et 99/100) : résultats identiques dans tous les cas.

Nature du problème : la post-condition $a^2 \leq n < (a+1)^2$ caractérise exactement la **racine carrée entière** (ou partie entière de la racine carrée) de n — le plus grand entier dont le carré ne dépasse pas n . Un nom pertinent : `racine_carree_entiere` (équivalent de `math.isqrt` en Python).

Exercice 9 — complément sur les ternaires ★★ (avancé)

1) Pré-condition :

```
assert n >= 0, 'Pre-condition'
```

2) a) Fonction auxiliaire `est_nombre_chiffres` :

```
def est_nombre_chiffres(n, k):
    if n == 0:
        return k == 1
    return 10**(k-1) <= n < 10**k
```

b) Post-condition déduite (via la fonction auxiliaire, comme suggéré dans les indications des rappels) :

```
assert est_nombre_chiffres(n, k), 'Post-condition'
```

3) Même post-condition, réécrite sous forme de ternaire (sans fonction auxiliaire) :

```
assert (k == 1 if n == 0 else 10**(k-1) <= n < 10**k), 'Post-condition'
```

4) Fonction complète (n n'est pas modifié : on travaille sur une copie locale m) :

```
def nombre_chiffres(n):
    assert n >= 0, 'Pre-condition'
    m = n
    k = 1
    while m >= 10:
        m //= 10
        k += 1
    assert (k == 1 if n == 0 else 10**(k-1) <= n < 10**k), 'Post-condition'
    return k
```

Vérifiée par exécution sur $n = 0, 1, 7, 9, 10, 42, 99, 100, 999, 1000, 12345$: k coïncide à chaque fois avec $\text{len}(\text{str}(n))$.

5) Généralisation aux entiers relatifs : on réutilise l'idée du code de l'exercice 2 (négation puis même calcul), en réappliquant nombre_chiffres à la valeur absolue :

```
def nombre_chiffres_relatif(n):
    return nombre_chiffres(abs(n))
```

Vérifiée sur des entiers négatifs et positifs (ex. $-42, -1, 0, 1, 42, -12345, 12345$) : le nombre de chiffres retourné coïncide bien avec celui de $|n|$.

À retenir

- **Jeu de test** : toujours viser le maximum de cas distincts (vide, singleton, cas général, cas limites, égalité stricte vs large) — l'objectif est de *mettre en défaut* le code, pas de le rassurer.
- **Boîte noire vs boîte blanche** : en boîte noire (fonction non fournie), on écrit seulement les jeux de test ; en boîte blanche (code fourni), on peut cibler précisément les branches et le nombre d'itérations du code pour choisir des valeurs qui les couvrent toutes.
- **Copie vs effet de bord** : bien distinguer une fonction qui retourne une nouvelle valeur sans toucher à ses paramètres, d'une fonction qui modifie ses paramètres en place et ne retourne rien (exercice 3).
- **Spécification** : type + pré-condition (sur les paramètres) + post-condition (sur le retour) ; la pré-condition peut elle-même s'appuyer sur une fonction auxiliaire (exercice 7) et la post-condition s'écrire de manière équivalente avec ou sans ternaire (exercice 9).
- **Ne jamais modifier les paramètres** d'une fonction dont on vérifie une post-condition : sinon la post-condition perd son sens (travailler sur une copie locale si nécessaire, comme $m = n$).